Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server • A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls **4 Marks**

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack) • How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file
- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
bufsize bytes. Returns the number of read() calls made, or -1 on error. */ long
copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];     int in = open(src, O_RDONLY);     if
(in < 0) return -1;     int out = open(dst, O_WRONLY | O_CREAT
| O_TRUNC, 0644);     if (out < 0) { close(in); return -1; }
long calls = 0;     ssize t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
//   stop when read returns 0 (end of file) or -1; write exactly the n bytes that     //
were read

    close(in);
close(out);
    return calls; }

int main(void) {
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);     fclose(f);

    size_t sizes[] = {1, 512, 4096};     for (int i = 0; i < 3; i++)
printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
copy_file("records.dat", "copy.dat", sizes[i]));
unlink("records.dat");     unlink("copy.dat");     return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls buffer
512 bytes:    21 read() calls buffer 4096
bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
// TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
//   status N" using WEXITSTATUS



    printf("shell: running job 2\n");     fflush(stdout);     pid = fork();     if
(pid == 0) _exit(3);                  /* a child that fails with code 3 */
waitpid(pid, &status, 0);     printf("job 2 exit status %d\n",
WEXITSTATUS(status));     return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0 shell: running job 2 job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner • The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare **4 Marks**

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan **4 Marks**

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN** → **MODIFY** → **or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4

Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3
Task	**Bloom's Level**	**Marks**
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

OPERATING SYSTEMS ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
int arrival[N] = {0, 1, 2, 3};     int burst[N] =
{8, 4, 9, 5};     int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
for (int i = 0; i < N; i++) left[i] = burst[i];     int time = 0, done = 0;
queue[tail++] = 0;     added[0] = 1;     printf("Gantt:");     while (done <
N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
//   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;     printf("\nProcess   TAT   WT\n");
for (int i = 0; i < N; i++) {          int tat = finish[i] -
arrival[i], wt = tat - burst[i];         tt += tat;         tw +=
wt;         printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23)
P3(23-26)
Process   TAT   WT
P1         23   15
P2         15   11
P3         24   15
P4         18   13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */ int
main(void)
{
    int fd[2];     if (pipe(fd) == -1) { perror("pipe"); return 1; }
pid_t pid = fork();     if (pid == 0) {         close(fd[1]);
/* child only reads */         int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
//   closed the pipe); keep the count, the sum and the maximum


        printf("analyser: %d readings, average %.1f, max %d\n", count,
count ? (double)sum / count : 0.0, max);         close(fd[0]);
fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                               /* parent only writes */     int
readings[] = {31, 33, 36, 34, 41, 38};     for (int i = 0; i < 6; i++)         if
(write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
close(fd[1]);                               /* signals end of data to the child */
waitpid(pid, NULL, 0);     printf("reader: done\n");     return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41 reader:
done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0; sem_t empty,
full; pthread_mutex_t lock =
PTHREAD_MUTEX_INITIALIZER; long consumed_sum = 0;

void *producer(void *arg) {
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
//   circularly, unlock, then post full

    }
    return NULL; }

void *consumer(void *arg) {
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
//   out circularly, unlock, post empty and add the item to consumed_sum

    }
    return NULL; }

int main(void)
{
    sem_init(&empty, 0, SIZE);     sem_init(&full, 0, 0);     pthread_t p, c;
pthread_create(&p, NULL, producer, NULL);     pthread_create(&c, NULL, consumer,
NULL);     pthread_join(p, NULL);     pthread_join(c, NULL);     printf("items
consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
sem_destroy(&empty);
sem_destroy(&full);     return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2}; int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2},
{2, 2, 2}, {4, 3, 3}}; int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2},
{2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */ int
is safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
memcpy(work, avail, sizeof work);     while
(count < P) {         int found = 0;
for (int p = 0; p < P; p++) {             if
(finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
//   finish: add alloc[p] to work, mark finish[p], append p to seq and set             //
found = 1

        }         if (!found)
return 0;
    }
    return 1; }

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)         if (req[r] > maxm[p][r] -
alloc[p][r] || req[r] > avail[r]) {             printf("P%d request:
must wait (not available)\n", p);             return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
int seq[P];     if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
printf("P%d request: denied (unsafe)\n", p);     }
}

int main(void)
{
    int seq[P];     if
(is_safe(seq)) {
        printf("initial state safe:");         for (int i =
0; i < P; i++) printf(" P%d", seq[i]);         printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
request(4, (int[R]){3, 3, 0});
request(0, (int[R]){0, 2, 0});     return
0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available) P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8

Q3	Threads	K5 – Evaluate	6
Question	**Assessed Area**	**Bloom's Level**	**Marks**
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend

OPERATING SYSTEMS ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).

- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks You

must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation You

must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating • Explain position-independent code

Experimental Investigation You

must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks You

must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

Evaluation You

must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation You

must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */ int
page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
for (int i = 0; i < 6; i++) {         unsigned a =
logical[i];
        // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
        //   PAGE_SIZE. If p is outside the table print "invalid address"; if the frame
//   is -1 print "page fault"; otherwise print the physical address frame *         //
PAGE_SIZE + d

    }
    return 0;
}
```

Expected output:

```
   0 -> page 0 offset    0 : physical 5120
1500 -> page 1 offset  476 : physical 2524
3100 -> page 3 offset   28 : physical 7196
6200 -> page 6 offset   56 : physical 3128
9000 -> page 8 offset  808 : invalid address
2049 -> page 2 offset    1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks You

must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation You

must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation You

must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes • Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks You

must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation You

must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation You

must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

> *Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.*

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks You

must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO

- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it • Compare the algorithms for a workload with strong locality

Experimental Investigation You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

Which page-replacement policy would you recommend for the web server? Use your measurements.

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
return -1; }

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
for (int i = 0; i < len; i++) {         if
(find(f, used, ref[i]) >= 0) continue;
faults++;
        if (used < frames) f[used++] = ref[i];         else {
f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults; }

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
for (int i = 0; i < len; i++) {         int k =
find(f, used, ref[i]);         if (k >= 0) {
last[k] = i; continue; }         faults++;
        // TODO 1: If a frame is free use it; otherwise replace the frame whose last use
//   (last[]) is the oldest. Record last[] = i for the frame used

    }
    return faults; }

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;     for (int i = 0; i <
len; i++) {         if (find(f, used, ref[i]) >= 0) continue;
faults++;         if (used < frames) { f[used++] = ref[i];
continue; }         int victim = 0, farthest = -1;         for
(int j = 0; j < frames; j++) {
            int next = len;                  /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults; }

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
int len = 20;
    for (int fr = 3; fr <= 4; fr++)         printf("%d frames: FIFO %d, LRU %d,
Optimal %d\n", fr, fifo(ref, len, fr),                lru(ref, len, fr),
optimal(ref, len, fr));     int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
fifo(b, 12, 3), fifo(b, 12, 4));     return 0;
}
```

Expected output:

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
```

```
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management **4 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks You

must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation You

must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation You

must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles **10 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock()) • Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator • Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

SIMATS ENGINEERING OPERATING SYSTEMS ASSIGNMENT 4 – CO4

OPTIMISING FILE STORAGE AND DISK ACCESS

Course: Operating Systems (CSA04)
Course Outcome: CO4 – Optimise various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance (file systems, file allocation methods, modern file systems, distributed and cloud-based storage, disk scheduling and device management)
Nature of Assignment: Individual
Duration: One Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.

- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment4** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to **choose how files are stored and how the disk is accessed by reasoning about the workload**, and to defend the choice with calculations and measurements.

You are expected to demonstrate the following progression:

> **Interpret the Workload → Identify the Operations → Compare the Methods → Calculate the Cost → Implement → Measure → Evaluate → Recommend**

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. All code must compile with gcc -std=c11 -Wall -Wextra.

TASK 1 – FILE-SYSTEM DESIGN REVIEW

Bloom's Level: K5 – Evaluate

Evaluate a Proposed File Organisation 6 Marks

A college proposes to store every student's files on a shared server as follows: one flat directory for all 12,000 students, files named only by the student's choice, contiguous allocation for every file, and no access control because "students will not look at each other's files".

Task 1.1 – Evaluate the Proposal | 2 Marks

Identify at least **five weaknesses**. Examine aspects such as:

- Name clashes in a single directory
- Search time for a file among 12,000 students
- File growth under contiguous allocation
- Protection and privacy
- Backup and recovery
- Sharing files for group projects

Task 1.2 – Improve the Design | 3 Marks

Design a better organisation: the directory structure (tree or acyclic graph), naming rules, file attributes, access-control lists or permission bits, and links for shared project folders.

Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, explain how your design protects privacy while still allowing sharing.

Alternative Concept and Complexity Analysis

If the student files were stored in a cloud service such as Google Drive or OneDrive instead of a shared server with its own directory structure, what would the solution be? Evaluate the time to find and open a file, the storage used (including copies), and the control the college keeps over privacy.

Evidence Expected

A reasoned evaluation, an improved file-system organisation and a concise justification.

TASK 2 – FROM WORKLOAD TO FILE-ALLOCATION METHOD

Bloom's Level: K5 – Evaluate

Storage for a Video-Surveillance Recorder

8 Marks

A network video recorder stores footage from 32 cameras. Files grow continuously while recording, are read mainly from start to end, are sometimes searched by time (random access), and are deleted after 30 days.

Task 2.1 – Identify the Operations | 2 Marks

Identify the file operations, their frequencies and the access pattern (sequential or direct).

Task 2.2 – Compare the Allocation Methods | 2 Marks

Compare contiguous, linked (including FAT) and indexed allocation for sequential access, direct access, file growth, fragmentation and reliability.

Task 2.3 – Calculate the Cost | 3 Marks

For a 4 GB file with 4 KB blocks, calculate the number of disk reads needed to reach the last block with each method, and the maximum file size supported by an inode with 12 direct, 1 single-, 1 double- and 1 triple-indirect pointer (4-byte block addresses).

Task 2.4 – Free-Space Management | 1 Mark

Compare a bit vector, a linked free list and grouping/counting for a 4 TB disk, and calculate the size of the bit vector.

Complete the Code

Complete TODO 1 below using first-fit contiguous allocation, and execute the program.

```
#include <stdio.h>
#include <string.h>

#define BLOCKS 32

char disk[BLOCKS];                          /* '.' = free, otherwise the file letter */

/* Contiguous allocation, first fit: returns the start block or -1. */ int
alloc_contiguous(char file, int size)
{
    // TODO 1: Scan the disk for the first run of size consecutive free blocks; mark
    //   them with the file letter and return the start block, or return -1 if no run is
//   long enough

}

/* Indexed allocation: one index block plus any free data blocks. */ int
alloc_indexed(char file, int size, int index[])
{
    int got = 0, idx = -1;     for (int b = 0; b < BLOCKS && got <= size; b++) {
if (disk[b] != '.') continue;         if (idx < 0) idx = b;                /* first
free block holds the index */         else index[got++] = b;         if (got == size)
break;
    }
    if (idx < 0 || got < size) return -1;     disk[idx] = (char)(file + 32);
/* lowercase letter = index block */     for (int i = 0; i < size; i++)
disk[index[i]] = file;     return idx; }

void release(char file)
{     for (int b = 0; b < BLOCKS; b++) if (disk[b] == file) disk[b] = '.';
}

int main(void)
{
    memset(disk, '.', BLOCKS);
alloc_contiguous('A', 6);
alloc_contiguous('B', 8);
alloc_contiguous('C', 5);
alloc_contiguous('D', 7);
release('B');     release('D');
    printf("%.32s\n", disk);     printf("E (10 blocks) contiguous: start %d\n",
alloc_contiguous('E', 10));     int index[16];
    int ib = alloc_indexed('E', 10, index);     printf("E (10 blocks)
indexed: index block %d, data blocks", ib);     for (int i = 0; i < 10;
i++) printf(" %d", index[i]);     printf("\n%.32s\n", disk);     return
0;
}
```

Expected output:

```
AAAAAA........CCCCC.............
E (10 blocks) contiguous: start 19
E (10 blocks) indexed: index block 6, data blocks 7 8 9 10 11 12 13 29 30 31
AAAAAAeEEEEEEECCCCCEEEEEEEEEEEEE
```

Explain why file E (10 blocks) cannot be stored contiguously although 21 blocks are free, and how indexed allocation solves the problem.

Alternative Concept and Complexity Analysis

If the recorder used linked allocation with a File Allocation Table (FAT) held in memory instead of indexed allocation, what would the solution be? Evaluate the time to reach block k of a file (O(k) look-ups in the FAT versus O(1) or a few index reads), the memory used by the FAT for a 4 TB disk, and the reliability if the FAT is damaged.

Evidence Expected

An operation analysis, a justified allocation method, correct cost and maximum-size calculations and a free-space comparison.

TASK 3 – MODERN FILE SYSTEMS

Bloom's Level: K5 – Evaluate

Choosing a File System for Different Devices 9 Marks

Task 3.1 – Compare Modern File Systems | 3 Marks

Compare FAT32, NTFS, ext4, XFS, Btrfs, ZFS and APFS for maximum file and volume size, journaling, snapshots, checksums and copy-on-write.

Task 3.2 – Journaling and Consistency | 2 Marks

Explain what happens to the file system when power fails during a write, and how journaling and copy-on-write keep it consistent. Compare the journaling modes of ext4 (journal, ordered, writeback).

Task 3.3 – Choose for Each Device | 3 Marks

Choose the file system for a pen drive used on Windows and Linux, a Linux database server, a laptop and a backup server that needs snapshots. Justify each choice.

Task 3.4 – Inspect a Real File System | 1 Mark

Use stat, df -i, ls -i and tune2fs -l or dumpe2fs -h on Linux to show inodes, block size and free space.

Alternative Concept and Complexity Analysis

If the database server used a copy-on-write file system such as Btrfs or ZFS instead of ext4 with journaling, what would the solution be? Evaluate the write time for small random updates, the extra space used by snapshots and checksums, and the speed of recovery after a power failure.

Evidence Expected

A comparison table, a correct explanation of consistency mechanisms, justified choices and command evidence.

TASK 4 – DISK SCHEDULING ALGORITHMS

Bloom's Level: K5 – Evaluate

Optimising Requests on a Records Server

7 Marks

The disk of a records server (cylinders 0–199) has its head at cylinder 53 and the request queue 98, 183, 37, 122, 14, 124, 65, 67.

Task 4.1 – Calculate | 2 Marks

Calculate the order of service and the total head movement for FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK.

Task 4.2 – Simulate | 2 Marks

Implement the algorithms and measure the average head movement and the maximum waiting time for 10,000 random requests with light, medium and heavy load.

Task 4.3 – Evaluate Fairness | 2 Marks

Evaluate which algorithms can starve requests, and why C-SCAN gives a more uniform waiting time than SCAN.

Task 4.4 – Solid-State Drives | 1 Mark

Explain why Linux uses the “none” or “mq-deadline” scheduler for SSDs, and check the scheduler of your disk (/sys/block/<disk>/queue/scheduler).

Complete the Code

Complete TODO 1 below to implement SSTF, and execute the program.

```
#include <stdio.h>
#include <stdlib.h>

#define N 8
#define MAXCYL 199

int req[N] = {98, 183, 37, 122, 14, 124, 65, 67};

int cmp(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }

void show(const char *name, const int order[], int n, int head, int extra)
{
    int total = 0, pos = head;
printf("%-6s %d", name, head);     for
(int i = 0; i < n; i++) {
printf(" %d", order[i]);         total
+= abs(order[i] - pos);         pos =
order[i];
    }
    printf("  -> %d cylinders\n", total + extra);
}

int main(void)
{     int head = 53, order[N +
2];

    show("FCFS", req, N, head, 0);

    /* SSTF: always serve the nearest pending request */
int done[N] = {0}, pos = head;     for (int k = 0; k < N;
k++) {
          // TODO 1: Find the pending request (done[i] == 0) with the smallest distance
      //   abs(req[i] - pos), mark it done, store it in order[k] and move pos to it


    }
    show("SSTF", order, N, head, 0);

    /* SCAN (towards cylinder 0 first, like an elevator) */     int
s[N];     for (int i = 0; i < N; i++) s[i] = req[i];     qsort(s, N,
sizeof s[0], cmp);     int n = 0, i;     for (i = N - 1; i >= 0; i--)
if (s[i] < head) order[n++] = s[i];     order[n++] = 0;     for (i =
0; i < N; i++) if (s[i] > head) order[n++] = s[i];     show("SCAN",
order, n, head, 0);

    /* C-SCAN (towards the last cylinder, then jump back to 0) */
n = 0;
    for (i = 0; i < N; i++) if (s[i] > head) order[n++] = s[i];     order[n++]
= MAXCYL;     int part = 0, p = head;     for (int k = 0; k < n; k++) { part +=
abs(order[k] - p); p = order[k]; }     int rest[N], m = 0;     for (i = 0; i <
N; i++) if (s[i] < head) rest[m++] = s[i];     int back = 0; p = 0;
    for (int k = 0; k < m; k++) { back += abs(rest[k] - p); p = rest[k]; }
    printf("C-SCAN %d ... %d, jump to 0 ... %d  -> %d cylinders (+%d for the jump)\n",
head, MAXCYL, rest[m - 1], part + back, MAXCYL);     return 0;
}
```

Expected output:

```
FCFS   53 98 183 37 122 14 124 65 67  -> 640 cylinders
SSTF   53 65 67 37 14 98 122 124 183  -> 236 cylinders
SCAN   53 37 14 0 65 67 98 122 124 183  -> 236 cylinders
C-SCAN 53 ... 199, jump to 0 ... 37  -> 183 cylinders (+199 for the jump)
```

Add a stream of requests near cylinder 60 arriving every millisecond and evaluate what happens to the request at cylinder 183 under SSTF.

Alternative Concept and Complexity Analysis

If the server used LOOK or C-LOOK (which turn at the last request instead of the end of the disk) instead of SCAN and C-SCAN, what would the solution be? Evaluate the total head movement for the given queue, the average and maximum waiting time, and the extra work needed to find the last request (O(n) or a sorted queue).

Evidence Expected

Correct calculations, measured simulation results, a reasoned fairness evaluation and evidence from a real system.

TASK 5 – DISTRIBUTED AND CLOUD-BASED STORAGE

Bloom's Level: K5 – Evaluate

7 Marks

A university wants all student and staff files to be available from any campus and from home.

Task 5.1 – Compare the Options | 3 Marks

Compare NFS, a distributed file system such as HDFS or Ceph, and cloud object storage such as Amazon S3 or Google Cloud Storage for consistency, availability, performance and cost.

Task 5.2 – Replication and Consistency | 2 Marks

Explain how replication (for example 3 copies in HDFS) protects data, and the trade-off between strong and eventual consistency.

Task 5.3 – Device Management | 2 Marks

Explain how the OS manages devices for storage: device drivers, buffering, caching, spooling and the block-device interface.

Alternative Concept and Complexity Analysis

If the university used one large NFS server instead of a replicated distributed file system, what would the solution be? Evaluate the read and write times as the number of users grows, the storage used (one copy versus three replicas), and the availability when one server fails.

Evidence Expected

A justified comparison of storage options, a correct explanation of replication and consistency, and of device management.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Storage Design for a Video-Streaming Start-up

13 Marks

A start-up streams lecture videos to 200,000 students. Each video is 300 MB to 2 GB, is uploaded once, and is read many thousands of times, mostly in the first week. Students jump to any minute

of a video. The company has a budget for a few servers with hard disks and SSDs, and may also use cloud storage.

You are not given the design.

Stage 1 – Interpret the Workload | 2 Marks

Identify the operations, access patterns and performance targets. State the requirements as testable statements.

Stage 2 – Propose the Storage Architecture | 2 Marks

Propose where videos are stored (SSD, hard disk, cloud), the file system and the allocation method.

Stage 3 – Identify the Critical Design Decisions | 2 Marks

Decide the block size, the disk scheduler, the caching of popular videos and the replication factor.

Stage 4 – Design and Calculate | 4 Marks

Present the design with calculations of capacity, throughput and the number of disk reads to start playing at any minute.

Stage 5 – Implement and Demonstrate | 3 Marks

Simulate the workload (for example with your allocation and disk-scheduling programs or with fio) and report the measured results for two alternative designs.

Alternative Concept and Complexity Analysis

If all videos were stored in cloud object storage and delivered through a content-delivery network instead of on the company's own servers, what would the solution be? Evaluate the time to start playback for students in different cities, the storage and transfer cost per month, and the effort of running the system.

Evidence Expected

A complete storage design with calculations, a simulation of two alternatives and an evidence-based choice.

MARK DISTRIBUTION

Task	Competency Assessed	Bloom's Level	Marks
Task 1	Evaluate and improve a file-system organisation	K5 – Evaluate	6
Task 2	Choose a file-allocation method from a workload	K5 – Evaluate	8
Task 3	Evaluate modern file systems	K5 – Evaluate	9
Task 4	Optimise disk scheduling	K5 – Evaluate	7
Task 5	Distributed and cloud storage, device management	K5 – Evaluate	7
Task 6	Independently design a storage system	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, script and configuration file, with input files, output screenshots and a README.md.

- Report containing evaluations, comparison tables, calculations, pseudocode and measurements
- C source code for Tasks 2, 4 and 6
- Command outputs and simulation results

Assessment Verification

Students may be asked to explain any design decision, repeat a calculation with new data, or change the design during a brief verification discussion/viva.

OPERATING SYSTEMS ASSIGNMENT 5 – CO5

LINUX, CONTAINERS AND VIRTUALISATION FOR EFFICIENT OPERATIONS

Course Outcome – CO5: Analyse the components and functionalities of Linux operating systems and virtualisation environments to enhance their operational efficiency (I/O systems, Linux systems, containerisation, virtualisation, system administration, cloud set-up, networking services, and tools such as Docker, Kubernetes and virtualisation platforms)
Nature: Individual
Duration: One Week **Total**
Marks: 50

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment5** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assessment Components

PART A – Design: 25 Marks
PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Modern services run on Linux, inside virtual machines and containers, in the cloud. The way the operating system, the hypervisor and the container platform are set up decides how many users a service can support, how quickly it recovers from failures and how much it costs.

This assignment investigates a fundamental question:

> **How should a service be deployed on Linux, virtual machines and containers so that it runs efficiently, scales with demand and recovers from failure?**

You are expected to demonstrate the following progression:

> **Understand Linux → Administer → Virtualise → Containerise → Orchestrate → Measure → Decide**

Simply obtaining a service that runs is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

Case Study — Deploying the University Result Portal

The university publishes semester results on a web portal. For most of the year it has a few hundred users a day, but on result day 150,000 students log in within two hours. The portal (a Python web application and a database) currently runs on one physical Linux server that crashes on result day. The university has a small private server room and an account with a public cloud provider.

PART A – DESIGN — 25 Marks

All pseudocode must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode.

TASK A1 – EVALUATE THE LINUX SYSTEM COMPONENTS — 7 Marks

Bloom's Level: K5 – Evaluate

A1.1 – Linux Architecture | 2 Marks

Explain the Linux kernel, system libraries, shell and utilities, and how the kernel manages processes, memory, files, devices and the network.

A1.2 – Process and Memory Management in Linux | 3 Marks

Evaluate the Linux Completely Fair Scheduler (CFS), the page cache and swapping, and the /proc and /sys file systems as tools for observing the system.

A1.3 – Linux File System and Permissions | 2 Marks

Explain the Linux directory hierarchy, the virtual file system (VFS), and the permission model (owner, group, others; setuid; sudo).

Alternative Concept and Complexity Analysis

If the portal server ran Windows Server instead of Linux, what would the solution be? Evaluate the memory and disk used by the base system, the time to automate administration tasks, and the licence cost per server.

Evidence Expected

A correct description of Linux components and a reasoned evaluation of its scheduling, memory and file management.

TASK A2 – DESIGN THE SYSTEM ADMINISTRATION 5 Marks

Bloom's Level: K5 – Evaluate

A2.1 – Users, Services and Logs | 2 Marks

Design the user accounts and groups, the systemd services and the log monitoring for the portal server.

A2.2 – Networking Services | 2 Marks

Design the networking set-up: IP addressing, DNS, a reverse proxy (Nginx), firewall rules (ufw or iptables) and SSH access with keys.

A2.3 – Automation | 1 Mark

Identify the routine tasks that should be automated with shell scripts and cron (backups, log rotation, disk-space alerts).

Alternative Concept and Complexity Analysis

If the server were configured with a configuration-management tool such as Ansible instead of hand-written shell scripts, what would the solution be? Evaluate the time to set up 1 server and 20 servers, the risk of differences between servers, and the effort to learn and maintain each approach.

Evidence Expected

A complete administration and networking design with an automation plan.

TASK A3 – EVALUATE VIRTUALISATION 5 Marks

Bloom's Level: K5 – Evaluate

A3.1 – Hypervisors | 3 Marks

Explain type-1 (bare-metal: KVM, VMware ESXi, Xen, Hyper-V) and type-2 (hosted: VirtualBox, VMware Workstation) hypervisors, full virtualisation and para-virtualisation, and hardware support (Intel VT-x, AMD-V).

A3.2 – Benefits and Costs | 2 Marks

Evaluate the benefits of virtual machines for the university (isolation, consolidation, snapshots, live migration) and their costs (overhead, memory per VM, licences).

Alternative Concept and Complexity Analysis

If a type-2 hypervisor such as VirtualBox were used instead of a type-1 hypervisor such as KVM, what would the solution be? Evaluate the overhead on CPU-bound and I/O-bound work, the memory used by the host operating system, and the suitability for production.

Evidence Expected

A correct explanation of virtualisation and a reasoned evaluation of its benefits and costs for the portal.

TASK A4 – EVALUATE CONTAINERISATION 4 Marks

Bloom's Level: K5 – Evaluate

A4.1 – How Containers Work | 2 Marks

Explain how containers use Linux namespaces and control groups (cgroups), and how a container image is built in layers.

A4.2 – Containers versus Virtual Machines | 2 Marks

Compare containers and virtual machines for start-up time, memory use, isolation and security, and decide where each should be used for the portal.

Alternative Concept and Complexity Analysis

If the portal and the database were each run in their own virtual machine instead of in containers, what would the solution be? Evaluate the start-up time (minutes versus seconds), the memory used per instance (a full guest OS versus shared kernel) and the strength of isolation.

Evidence Expected

A correct explanation of namespaces, cgroups and image layers, and a justified comparison.

TASK A5 – DESIGN THE CLOUD DEPLOYMENT AND ORCHESTRATION 4 Marks

Bloom's Level: K5 – Evaluate

A5.1 – Cloud Set-up | 2 Marks

Compare IaaS, PaaS and SaaS, and design a cloud set-up for result day (virtual machines or a managed Kubernetes service, a load balancer, a managed database).

A5.2 – Kubernetes | 2 Marks

Explain pods, deployments, services, replicas, resource requests and limits, and the horizontal pod autoscaler, and how they let the portal scale and recover from failures.

Alternative Concept and Complexity Analysis

If the cloud deployment used fixed virtual machines sized for result day instead of Kubernetes with an autoscaler, what would the solution be? Evaluate the response time on result day, the resources left idle on the other 364 days, and the cost.

Evidence Expected

A justified cloud design and a correct explanation of how Kubernetes provides scaling and recovery.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION 25 Marks

Part B assesses whether your design reasoning survives contact with real systems. You are expected to:

Implement → Deploy → Load → Measure → Compare → Interpret

Use a Linux machine or virtual machine. Record the commands you run and their outputs. Execution screenshots alone are insufficient.

TASK B1 – ADMINISTER THE LINUX SERVER 7 Marks

Bloom's Level: K5 – Evaluate

B1.1 – Observe the System | 2 Marks

Use ps, top or htop, free, vmstat, iostat, df and ss to record the processes, memory, disk and network use of your server, and explain what each value means.

B1.2 – Automate | 2 Marks

Write shell scripts for log analysis and disk-space alerts and schedule them with cron. Show their output.

B1.3 – Services and Network | 2 Marks

Run the portal (or a sample web application) as a systemd service behind Nginx, with firewall rules that allow only HTTP, HTTPS and SSH.

B1.4 – Compare with Your Design | 1 Mark

Compare what you built with your Part A design and explain any change.

Complete the Code

Complete TODO 1 and TODO 2 below using awk, sort and uniq, and execute the script with bash on Linux.

```
#!/usr/bin/env bash
# Summarise a web-server access log: requests per status code and the busiest client.
set -euo pipefail

log=$(mktemp) cat >
"$log" <<'LOG'
10.0.0.5 - - [08/Oct/2026:10:01:02] "GET /index.html" 200 512
10.0.0.7 - - [08/Oct/2026:10:01:05] "GET /login" 200 1024
10.0.0.5 - - [08/Oct/2026:10:01:09] "POST /login" 302 0
10.0.0.9 - - [08/Oct/2026:10:02:11] "GET /admin" 403 128
10.0.0.5 - - [08/Oct/2026:10:02:15] "GET /report.pdf" 200 40960
10.0.0.7 - - [08/Oct/2026:10:03:01] "GET /missing" 404 64
10.0.0.5 - - [08/Oct/2026:10:03:30] "GET /index.html" 200 512 LOG

echo "Requests per status code:"
# TODO 1: Print each status code (field 7) with its count, sorted by code, in the form " #
200 4" using awk, sort and uniq



echo "Busiest client:"
# TODO 2: Print the IP address (field 1) with the most requests and its count, in the #
form "  10.0.0.5 4"



echo "Total bytes sent: $(awk '{s += $8} END {print s}' "$log")" rm -
f "$log"
```

Expected output:

```
Requests per status code:
  200 4
  302 1
403 1
404 1
Busiest client:
  10.0.0.5 4
Total bytes sent: 43200
```

Schedule the script with cron to run every hour and show the crontab line you used.

Alternative Concept and Complexity Analysis

If the logs were collected and analysed by a central log system (for example journald with a log server or the ELK stack) instead of a shell script on each server, what would the solution be? Evaluate the time to find an error across 10 servers, the extra storage and memory used, and the set-up effort.

Evidence Expected

Command evidence, working scripts, a running service and a comparison with the design.

TASK B2 – CONTAINERISE THE PORTAL 5 Marks

Bloom's Level: K5 – Evaluate

- Write a Dockerfile for the portal, build the image and run it with docker run, mapping the port
- Run the portal and its database with Docker Compose, using a volume for the database data
- Limit the container's CPU and memory (--cpus, --memory) and show the effect with docker stats
- Inspect the image layers (docker history) and reduce the image size (for example a slim base image or a multi-stage build)
- Explain what each step shows about how containers work

Complete the Code

Complete TODO 1 to TODO 3 below. Build the image with docker build -t result-portal . and run it with docker run -p 8000:8000 result-portal.

```
# Container image for the campus result-portal web service (a small Flask app). FROM
python:3.12-slim

# TODO 1: Set the working directory inside the image to /app



# Install the dependencies first so that this layer is cached between builds.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# TODO 2: Copy the application source into the image and run it as a non-root user named #
portal



# TODO 3: Document that the service listens on port 8000 and start it with gunicorn on #
0.0.0.0:8000 using the app object in app.py
```

Expected result: the image builds without errors, docker ps shows the container running, and http://localhost:8000 shows the portal. Record the image size shown by docker images and the user shown by docker exec <container> whoami (it should be portal).

Alternative Concept and Complexity Analysis

If the image were built from a full Ubuntu base image instead of python:3.12-slim, or with a multi-stage build, what would the solution be? Evaluate the image size, the build time, the time to pull the image to a new server, and the number of packages that may contain security problems.

Evidence Expected

A working container set-up, resource-limit evidence, an optimised image and a reasoned explanation.

TASK B3 – COMPARE BARE METAL, VIRTUAL MACHINE AND CONTAINER 8 Marks

Bloom's Level: K5 – Evaluate

B3.1 – Deploy Three Ways | 2 Marks

Run the same web application directly on Linux, inside a virtual machine (VirtualBox or KVM) and inside a Docker container.

B3.2 – Measure | 2 Marks

Measure start-up time, memory used, image or disk size and requests per second (with ab, wrk or hey) for each.

B3.3 – Scale with Kubernetes | 2 Marks

Deploy the container to a local Kubernetes cluster (minikube or kind) with 3 replicas, delete one pod and show that it is recreated, and scale to 6 replicas under load.

B3.4 – Engineering Trade-Off | 2 Marks

Explain the results and state when each deployment method should be used.

Complete the Code

Complete TODO 1 and TODO 2 in the Kubernetes file deployment.yaml below. Apply it with kubectl apply -f deployment.yaml on minikube or kind.

```
# Kubernetes Deployment and Service for the result portal.
apiVersion: apps/v1
kind: Deployment
metadata:   name:
result-portal spec:
  # TODO 1: Run 3 replicas, and select the pods that carry the label app: result-portal

  template:
metadata:
labels:
        app: result-portal
spec:
      containers:         - name: portal
image: simats/result-portal:1.0
ports:
            - containerPort: 8000
          # TODO 2: Request 250m CPU and 256Mi memory, and limit the container to 500m
#   CPU and 512Mi memory

          readinessProbe:
httpGet:
              path: /health
port: 8000
---
apiVersion: v1 kind:
Service metadata:
name: result-portal
spec:   type:
LoadBalancer
selector:
    app: result-portal
ports:     - port: 80
targetPort: 8000
```

Expected result: kubectl get pods shows 3 pods in the Running state. After kubectl delete pod <name>, a new pod is created automatically. kubectl describe pod shows the requests and limits you set.

Alternative Concept and Complexity Analysis

If the portal were scaled with Docker Compose (docker compose up --scale portal=6) on one server instead of Kubernetes on several nodes, what would the solution be? Evaluate the time to recover from a failed container or server, the resources needed, and the maximum load each can handle.

Evidence Expected

Three working deployments, a measured comparison table, Kubernetes scaling and recovery evidence and a reasoned trade-off.

TASK B4 – FINAL ENGINEERING DECISION 5 Marks

Bloom's Level: K5 – Evaluate

The university will also host its learning-management system and online examinations on the same infrastructure, and wants result day to be handled without buying new servers.

B4.1 – Implement a Practical Extension | 2 Marks

Add a second service to your Kubernetes or Docker Compose set-up and show that both run with their own resource limits.

B4.2 – Estimate the Capacity | 1 Mark

Using your measured requests per second, estimate the number of containers or virtual machines needed for 150,000 students in two hours.

B4.3 – Production Decision | 2 Marks

Decide whether the university should use its own servers with virtual machines, its own servers with containers and Kubernetes, the public cloud, or a hybrid. Justify your decision with your measurements and cost estimates.

Alternative Concept and Complexity Analysis

If the university used only the public cloud (paying per hour) instead of its own servers with Kubernetes, what would the solution be? Evaluate the cost over a year with one result day per semester, the time to add capacity, and who controls the student data.

Evidence Expected

A working extension, a capacity estimate and an evidence-based production decision.

MARK DISTRIBUTION

Component	Task	Bloom's Level	Marks
Part A – Design	A1 – Linux System Components	K5 – Evaluate	7
	A2 – System Administration	K5 – Evaluate	5
	A3 – Virtualisation	K5 – Evaluate	5
	A4 – Containerisation	K5 – Evaluate	4
	A5 – Cloud and Orchestration	K5 – Evaluate	4
	Part A Total		**25**
Part B – Implementation	B1 – Linux Administration	K5 – Evaluate	7
	B2 – Containerisation	K5 – Evaluate	5
	B3 – Deployment Comparison	K5 – Evaluate	8
	B4 – Final Engineering Decision	K5 – Evaluate	5
	Part B Total		**25**
	TOTAL		**50**

SUBMISSION REQUIREMENTS

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

For **Part A**, submit the diagrams, comparison tables, designs and reasoning.

For **Part B**, submit shell scripts, Dockerfiles, Compose and Kubernetes files, commands with their outputs, measurements and interpretation.

Students are expected to use the same engineering workflow throughout the assignment:

> **UNDERSTAND → DESIGN → DEPLOY → LOAD → MEASURE → INTERPRET → DECIDE**

The key question is not simply “Does the portal run?” It is also:

> **Will it stay up on result day, recover from failures and use the university’s resources efficiently?**